ChainCert Infrastructure Deployment Models: Managed vs Consortium Setup and Current Prototype Implementation

Notes on which model is better: chaincert handles everything/all nodes, or we give a deployment node kit to the issuer tas sila maghandle ng node nila

1. Background

ChainCert is implemented on **Hyperledger Fabric**, which supports a **consortium-style blockchain architecture** where multiple organizations participate in a shared network. Each organization is represented by an **MSP (Membership Service Provider)** and can operate its own blockchain infrastructure such as peers and certificate authorities.

In the current prototype, however, all infrastructure is hosted and managed by the development team due to the absence of external client organizations. This raises an architectural question regarding the intended deployment model of the system.

Two possible deployment models are considered.

---

2. Model A — ChainCert-Managed Infrastructure

Description

In this model, **ChainCert operates the entire blockchain infrastructure**, including:

- Orderer nodes
- Peer nodes
- Fabric Certificate Authorities
- Channel management

- Identity enrollment

Organizations interact with the system **only through the ChainCert platform interface**, without managing any blockchain components themselves.

Example architecture:

```
ChainCert Infrastructure
│
├ Orderer Node
├ Peer Node (ChainCert)
├ Peer Nodes for organizations (hosted by ChainCert)
├ Fabric CA
└ Backend platform
```

Organizations such as universities simply register as issuers and use the dashboard to issue credentials.

Advantages

Simpler onboarding

Organizations do not need blockchain or DevOps knowledge.

Lower operational cost for organizations

No server maintenance is required.

Faster adoption

Institutions can use the system immediately after approval.

Easier network management

ChainCert maintains full control over upgrades, configuration, and security.

Disadvantages

Less decentralized

Infrastructure is centrally operated.

Reduced organizational control

Participating institutions rely on ChainCert for network operations.

Potential scalability concerns

All infrastructure must be maintained by the platform operator.

---

3. Model B — Consortium Model (Organization-Operated Nodes)

Description

In this model, **each organization operates its own blockchain node**.

ChainCert provides:

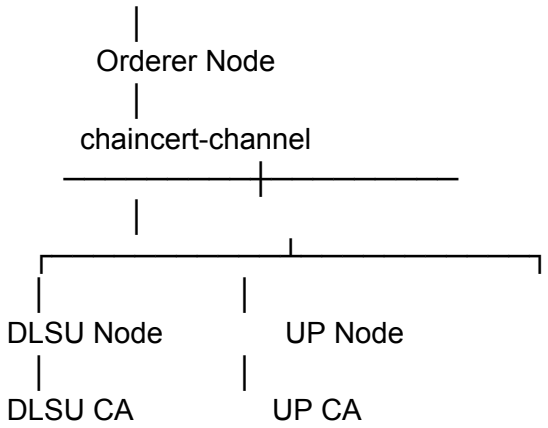
- the platform
- governance
- onboarding process
- deployment kit

Each organization deploys:

- its own **peer node**
- its own **Fabric CA**
- its own **MSP identities**

Example architecture:

ChainCert



Each organization becomes an independent participant in the blockchain network.

Advantages

True consortium architecture

Matches Hyperledger Fabric's intended design.

Organizational autonomy

Institutions control their own identities and infrastructure.

Higher trust model

No single party controls the entire network.

Better decentralization

Ledger validation is distributed across organizations.

Disadvantages

Higher technical complexity

Organizations must maintain blockchain infrastructure.

Requires DevOps knowledge

IT teams must manage nodes, certificates, and updates.

Slower onboarding

Organizations must deploy infrastructure before joining.

Higher operational costs

Servers and maintenance are required.

---

4. Onboarding Process (Consortium Model)

If organizations operate their own nodes, the onboarding process would be:

Step 1 — Issuer registration

Organization registers as a **root authority issuer** on the ChainCert platform.

Status:

pending_review

Step 2 — Document verification

ChainCert admin verifies submitted documents.

If approved:

deployment_required

Step 3 — Deployment kit distribution

ChainCert sends the organization a **deployment kit**, which includes:

- Docker compose files
- Fabric configuration files

- enrollment scripts
- channel artifacts
- chaincode package
- onboarding instructions

Step 4 — Organization deploys node

Organization performs the following:

1. Deploy Fabric CA
2. Enroll admin and peer identities
3. Start peer node
4. Join `chaincert-channel`
5. Install and approve chaincode

Step 5 — Organization informs ChainCert

After deployment, the organization informs ChainCert that the node is ready.

In the prototype, this step is **manual communication**.

In a production system, this would ideally be a **system submission step** where the organization provides:

- MSP ID
- peer endpoint
- connection profile
- org identity information

Step 6 — ChainCert verification

ChainCert verifies that:

- the organization joined the channel
- the peer node is operational
- chaincode approval is completed

Step 7 — Final activation

The issuer account status becomes:

active

The organization can now log in and issue credentials.

---

5. Prototype Implementation

In the current prototype:

- ChainCert operates all infrastructure.
- Nodes for multiple organizations are simulated within the same environment.
- Identities are enrolled through ChainCert-controlled processes.

Example setup:

ChainCert Server (Ubuntu)
│
├ Orderer Node
├ Peer Node (ChainCert)
├ Peer Node (DLSU)
├ Fabric CA

├ CouchDB
└ Backend + Frontend

Although centrally hosted, **the logical separation of organizations is preserved** through separate MSPs and identities.

---

6. Manual Node Registration (Prototype)

Since automated node verification is not implemented yet, organizations would **manually inform ChainCert** after deployment.

Example:

"Hello ChainCert Admin,

Our organization has completed node deployment and joined the chaincert-channel.
Please verify and activate our issuer account."

ChainCert then verifies and activates the account.

This manual process does **not violate system specifications**, as the architecture still supports proper node onboarding.

Future versions could implement a **Node Registration submission form**.

---

7. Final Verdict (Current Working Assumption)

Based on the system architecture and proposal:

Conceptual architecture

The system aligns more closely with a **consortium-style model**, where each organization may operate its own node.

Prototype implementation

For development practicality, **ChainCert currently hosts and manages the infrastructure centrally**.

Thus the final explanation is:

> The system architecture is designed to support a consortium deployment where each participating organization can operate its own blockchain node. However, the current prototype uses a centrally hosted infrastructure managed by ChainCert to simplify development and testing.

---

8. Current Status

For the purposes of the thesis:

- Both models are documented.
- The system supports consortium deployment.
- The prototype uses managed infrastructure.

Final architectural framing will be confirmed after consulting the project adviser.